

# IBD - TP Docker

## Exercice 1 - Manipulation basique de conteneurs

Le but de cet exercice est de comprendre comment fonctionne la création, la suppression et la modification d'un conteneur docker.

### 1. Afficher tous les conteneurs actuellement lancés sur votre machine.

```
docker ps
```

Cela n'affiche rien ? Cela veut dire qu'aucun container ne tourne, essayez de rajouter l'option « -a » à la commande

Cela permet de lister tous les conteneurs peu importe leur statuts (running, stopped.. ). Cela n'affiche toujours rien ? C'est normal si vous venez d'installer Docker 😊

### 2. Lancez un premier conteneur

Pour cela, rien de plus simple, exécutez la commande suivante :

```
docker run hello-world
```

Bravo! Vous maitrisez Docker! 🥳

Maintenant re-listez les conteneurs, est-ce que vous voyez le conteneur que vous venez d'exécuter ? Dans quel état est-il ?

Qu'en déduisez-vous ?

### 3. Arrêter un conteneur du terminal

Lancez un conteneur avec l'image nginx.

*(C'est quoi une image ?..)*

Une image c'est simplement le modèle à partir duquel est créer le conteneur. Tout à l'heure, nous avons créé un conteneur à partir de l'image hello-world. Cette dernière affiche un message dans la console puis se coupe.

La commande à exécuter est donc :

```
docker run nginx
```

Le conteneur est lancé mais vous n'avez la main sur la console. En effet il s'agit d'un serveur d'application, le serveur resta actif le temps qu'on ne le coupe pas donc vous n'aurez plus la main avant de « killer » ou stopper le conteneur

Ouvrez un autre terminal et entrez la commande suivante pour arrêter le conteneur :

```
docker ps -> récupérez l'id du conteneur en exécution
```

```
docker stop <id du conteneur>
```

ou

```
docker kill <id du conteneur>
```

Refaites un listing des conteneurs pour valider que rien ne tourne

#### 4. Lancer un conteneur détaché du terminal

##### Conteneur vs Nginx round 2

On va retenter de lancer un conteneur avec nginx, néanmoins ça serait vachement cool de pouvoir garder la main sur le terminal, l'option « -d » est faite pour ça !

```
docker run -d nginx
```

Cela nous permet de lancer un conteneur en mode détaché du terminal (un peu comme si on exécutait la commande en tâche de fond)

Comme toujours, n'hésitez pas à lister les conteneurs pour voir s'il est bien en train de tourner correctement

#### 5. Se connecter à un conteneur

*« Ok, nous pouvons lancer et supprimer des conteneurs c'est top !*

*Mais de base, c'est censé être des environnements virtuels, comme des VM, est-ce qu'on peut y accéder pour s'amuser dessus sans avoir peur de casser quelque chose ? »*

Oui ! Pour cela, on va partir d'une image d'Ubuntu

Exécutez la commande :

```
docker run -it ubuntu /bin/bash
```

L'objectif de cette commande est de créer un conteneur sur la base de la dernière image d'Ubuntu, puis de s'y connecter avec l'option -it (pour interactive terminal).

Le « /bin/bash » permet de sélectionner le répertoire de la machine sur lequel on se situera quand on s'y connectera.

Maintenant, vous êtes connectés au conteneur et vous pouvez dorénavant vous balader dans les fichiers de conf, installer des outils etc.. Tout comme sur un vrai PC !

#### 6. Se connecter à un conteneur déjà existant

Normalement, votre conteneur nginx tourne toujours (sinon relancez la commande du point n°4 pour l'exemple).

Vous pouvez vous y connecter également pour voir comment la machine est constituée.

En effet, si vous essayez de faire la commande « docker run -it nginx » vous verrez que vous n'aurez quand même pas la main sur le terminal

La commande à exécuter est

```
docker exec -it <id du conteneur nginx> /bin/bash
```

#### 7. Exposer un port d'un conteneur

Gardons notre conteneur nginx, pour rappel il s'agit d'un serveur d'application (un peu comme les serveurs Tomcat d'apache), le but de cette technologie est d'héberger et donner accès à une application.

Néanmoins, actuellement on a lancé le conteneur mais nous ne pouvons rien faire avec, c'est dommage !

Ça serait top si nous pouvions accéder à nginx depuis un navigateur internet de notre PC local (pas le conteneur en lui-même)

Pour cela, il faut mapper un port d'exposition au conteneur. Par défaut nginx expose le port 80.

Essayons la commande suivante :

```
docker run -d -p 8080:80 nginx
```

Cette dernière nous permet de lancer un conteneur nginx en synchronisant le port 8080 de notre PC local et le port 80 du conteneur.

Pour vérifier, ouvrez un Google Chrome ou autre navigateur, et allez à l'adresse suivante : <http://localhost:8080>

Si la page suivante s'affiche, vous avez accès à votre conteneur nginx, bravo !

# Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to [nginx.org](http://nginx.org).  
Commercial support is available at [nginx.com](http://nginx.com).

*Thank you for using nginx.*

## 8. Injecter des variables d'environnement

Parfois, il est nécessaire de paramétrer des variables d'environnements sur le conteneur (exemple : variabiliser des urls, des environnements etc..)

Pour cela :

```
docker run -d -e MA_VAR=TEST nginx
```

Pour vérifier l'installation de la variable d'environnement, connectez-vous au conteneur nginx et tapez la commande « env » pour afficher les variables d'environnement du conteneur. Logiquement, la variable est censée apparaître

## 9. Analyser conteneur

Si vous avez des problèmes ou besoin d'informations sur un conteneur, il existe plusieurs commandes permettant d'analyser votre conteneur

docker logs <id du conteneur> -> permet de voir les journaux d'exécution d'un conteneur

docker inspect <id du conteneur> -> permet de voir les informations techniques d'un conteneur

```
docker inspect --format '{{.Name}} {{.Config.Image}} {{.State.Status}}' <id du conteneur>
```

-> permet d'extraire certaines données du docker inspect

## 10. Supprimer tous les conteneurs pas utilisés

Pour finir, faisons un peu de ménage dans les conteneurs qu'on a utilisé et qui ne sont plus actifs.

`docker container prune` (avec l'option « -f » si vous voulez évitez la confirmation)

## Exercice 2 – Manipulation des images

Objectif : Créer une image sans dockerfile.

Lancer un conteneur à partir de la dernière image d'Ubuntu

### 1. Lister les images existantes

Comme pour les conteneurs, nous pouvons manipuler les images afin d'avoir un modèle de conteneur facilement montable et utilisable

Pour lister les images :

```
docker images
```

Vous pourrez voir les images que vous avez utilisé, notamment ubuntu, nginx et hello-world de l'exercice d'avant

### 2. Construire une image docker python à partir d'une d'ubuntu

Le but de cet exercice est de pouvoir adapter une image existante en fonction d'un besoin spécifique.

Lancez un conteneur à partir d'une image ubuntu et connectez-vous dessus

Une fois sur la machine du conteneur, installer python avec les commande suivantes :

```
apt-get update  
apt-get install python
```

Vous pouvez vérifier l'installation en tapant la commande « `python` », logiquement vous pourrez commencer à écrire des lignes python et à les exécuter

Quittez la machine (commande « `exit` »)

Ici, on a pris une image Ubuntu et installé python dessus, bien joué ! Maintenant, le but est de pouvoir créer une image docker par rapport à cette version de la machine pour pouvoir lancer un conteneur facilement à partir de cette nouvelle image

Exécutez la commande suivante :

```
docker commit <id du conteneur> mon_image_ubuntu_python
```

Bravo, vous avez créé votre première image, vérifiez que tout va bien en lançant une instance de l'image

```
docker run -it mon_image_ubuntu_python
```

### 3. Registry d'images

Docker est un outil largement utilisé, énormément d'images sont disponibles en open source.

Vous pouvez les retrouver sur le site de docker hub (<https://hub.docker.com>)

Il est possible de récupérer des images depuis le hub ou depuis un autre provider d'images (par exemple GCP) via la commande

```
docker pull <nom de l'image> -> pour les images sur le hub  
docker pull <lien vers l'image>
```

À l'inverse, il est également possible d'uploader des images sur docker hub

Pour cela, il faut bien sûr se créer un compte sur le site, puis de synchroniser son terminal avec ce compte

```
docker login
```

Puis envoyer l'image sur le registry

```
docker push <nom de l'image>:<tag> -> le tag est une version que vous renseignez, il est facultatif
```

### 4. Exporter/importer une image sans docker hub

Il est également possible d'exporter une image sans l'utilisation d'internet

Pour exporter une image pour la donner à votre collègue, vous pouvez exécuter la commande

```
docker save {nom de l'image} > {nom de fichier}.tar  
ou docker save --output {nom de fichier}.tar {nom de l'image} (si vous êtes sur windows)
```

Du point de vue de votre collègue, pour importer l'image

```
docker load < {nom de fichier}.tar  
ou docker load --input {nom de fichier}.tar (si vous êtes sur windows)
```

Et voilà ! Vous pouvez re-lister les images pour voir si elle a bien été importée

### 5. Faire le ménage dans les images

Pour supprimer les images non utilisées, la commande suivante est faite pour vous :

```
docker images prune
```

## Exercice 3 – Les volumes

Objectif : Manipuler un volume Docker

« C'est cool, on peut construire un conteneur mais tout ce que je fais dedans n'est pas sauvegardé.. Comment Je peux faire pour récupérer mes données qui sont générées dedans ? »

C'est tout le principe d'un volume, il s'agit d'une synchronisation entre un répertoire à l'intérieur du conteneur et un répertoire de la machine hôte

### 1. Lister tous les volumes présents sur votre machine.

Comme pour les conteneurs et les images, nous avons une ressource volume qu'on peut manipuler

Pour le listing, il s'agit de la commande

```
docker volume ls
```

On pourra y voir le driver et le nom du volume, pour plus d'informations, n'hésitez pas à faire un inspect d'un volume ☺

### 2. Créer un volume appelé html-nginx.

Pour cela, rien de plus simple

```
docker volume create html-nginx
```

Cette commande permet d'initialiser le volume que nous allons utiliser entre notre local et notre conteneur pour récupérer les données du conteneur

### 3. Lancer un conteneur nginx en synchronisant un répertoire

Maintenant que nous avons créé notre volume, il est temps de l'utiliser !

Choisissez un dossier dans votre machine hôte avec lequel vous allez synchroniser vos données

Tapez la commande suivante :

```
docker run -d -v <le lien de votre dossier>:/test nginx
```

Commande sur windows : `docker run -d --mount type=bind,source=<votre lien absolu de la machine hôte>,target=<liendans le conteneur> nginx`

Ici, nous synchronisons donc le répertoire que vous avez choisi avec un répertoire test qui se trouve dans le conteneur

Testons que tout fonctionne bien ☺

Connectez-vous au conteneur (pour rappel avec `docker exec -it <id du conteneur>`), allez dans le dossier test et créez un fichier

Si ce fichier est aussi créé dans le répertoire de la machine hôte, bravo vous avez bien créé votre volume !

### 4. Synchroniser plusieurs conteneurs

Il est également possible de créer une zone de partage entre plusieurs conteneurs dans le cas où 2 applications ont besoins des mêmes données ou s'ils doivent s'échanger des données

Comment faire ? -> avec l'option --mount  
Créez un volume shared-volume

Lancez un premier conteneur basé sur Ubuntu « monté » (mounted) sur ce volume

```
docker run -it --mount type=volume,source=shared-volume,target=/data ubuntu
```

Explications :

- Docker run -it -> vous commencez à avoir l'habitude ☺
- --mount source=shared-volume, target=/data -> on monte le conteneur sur le volume "shared-volume" et le répertoire du conteneur qui sera synchronisé sera le dossier « data »

Créez un fichier « test.txt » dans le dossier data de ce conteneur puis lancez un 2<sup>e</sup> conteneur avec la même commande qui ci-dessus pour monter un 2<sup>e</sup> conteneur sur le même volume

Vérifiez que vous voyez bien le test.txt dans le 2<sup>e</sup> conteneur

Vous êtes maintenant un pro de la synchro ☺

## 5. Faire le ménage des volumes

Comme d'habitude, on fait le ménage après nos tests avec cette fois ci la commande

```
docker volume prune
```

Si un volume est utilisé par un conteneur, il faudra d'abord le supprimer pour pouvoir supprimer le volume

## Exercice 4 – Les Dockerfiles

Objectif : Créer une image avec dockerfile

Les dockerfile sont un des outils les plus importants de l'utilitaire Docker. Ils permettent essentiellement de créer des images avec une approche de la méthodologie IaC (Infrastructure as Code)

### 1. Nous allons reprendre l'exemple de l'installation de python à partir d'une image ubuntu

Créez un répertoire quelconque (tips si vous manipulez tout en ligne de commande, « mkdir <nom du répertoire> » permet de créer un dossier)

Créez un fichier nommé Dockerfile (respectez bien le nom de fichier sinon docker ne va pas le reconnaître, il ne doit pas avoir d'extension)

Pour créer notre image comme dans l'exercice précédent, nous étions partis d'une image ubuntu

Dans le Dockerfile, faisons la même chose en écrivant :

```
FROM ubuntu
```

L'instruction FROM permet de dire quelle image nous prenons en référence

L'étape suivante est d'installer python sur cette version d'Ubuntu, pour ça on va exécuter une commande shell comme quand nous nous étions connectés sur le conteneur

RUN apt-get update && apt-get install -y python3

RUN nous permet d'exécuter du shell dans la création de l'image

Pour s'assurer que c'est bien installé, nous pouvons dire au Dockerfile d'exécuter une commande python dans le conteneur

```
ENTRYPOINT ["python3", "-c", "print('hello')"]
```

ENTRYPOINT est le point d'entrée du conteneur, c'est cette commande qui sera lancée quand on va run le conteneur à partir de cette image

Bravo, vous avez créé votre Dockerfile, trop fort !

## 2. Compiler le Dockerfile

Maintenant que vous avez créé le Dockerfile, il va falloir le compiler pour que Docker vous permette de construire l'image et de lancer un conteneur avec l'image de référence.

Pour cela, dans le terminal, placez-vous au même endroit que le Dockerfile et lancez la commande suivante

```
docker build -t mon_image_python:1.0.0 .
```

Explications :

- docker build pour compiler un Dockerfile
- -t pour donner un tag à cette image -> pour rappel il s'agit d'une version qu'on vient renseigner
- Le « . » à la fin est très important, il indique le répertoire où se situe le Dockerfile à compiler

Une fois cette étape lancée, l'installation de python va se faire et si tout se passe correctement, vous aurez votre image « mon\_image\_python:1.0.0 » avec laquelle vous pourrez construire de beaux conteneurs !

## 3. Ajouter de la configuration aux conteneurs

On a vu juste avant comment synchroniser des répertoires entre le local et le conteneur grâce aux volumes, mettons ça en place via le Dockerfile

Avant le ENTRYPOINT, inscrire la ligne suivante

```
VOLUME /data
```

De plus pour pas que le conteneur s'arrête instantanément pour qu'on puisse faire quelques tests, changez le ENBTRYPOINT pour le suivant

```
CMD ["tail", "-f", "/dev/null"] -> cela ouvre un flux continu vers un fichier, tant que le flux est ouvert, le conteneur ne se stoppera pas. Le CMD permet également d'exécuter un script shell
```



Lancez le conteneur avec la commande adapté comme vue dans l'exercice précédent

Dans un 2<sup>e</sup> terminal, allez-vous connecter sur le conteneur et créez un fichier texte dans le répertoire data, normalement vous aurez le même fichier dans le répertoire synchronisé avec le volume

TOP ! le volume fonctionne

On va faire la même chose avec les variables d'environnement

Avant le ENTRYPOINT, ajoutez la ligne suivante :

```
ENV MA_VAR=TEST
```

Ici pareil, lancez le conteneur et dans un 2<sup>e</sup> terminal (ou avec un -d) connectez-vous à ce dernier et affichez les variables d'environnement pour voir que tout s'est bien passé

Enfin la troisième chose que l'on va tester est l'exposition d'un port comme dans le premier exercice

Copier le contenu suivant dans votre Dockerfile

```
FROM nginx:latest
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

EXPOSE permet d'exposer notre application sur un port donné

Avec la commande `docker run -p 8080:80 <nom de l'image>`, nous pouvons nous connecter au contenu de l'application

## Exercice 5 – Gestion du réseau

Objectif : savoir connecter nos conteneurs à la machine hôte ou à d'autres conteneurs

### 1. Lister les réseaux configurés existants

```
docker network ls
```

Toujours, pareil faites un inspect sur un réseau listé pour avoir plus d'informations

### 2. Créer un réseau

Créons un réseau pour pouvoir permettre à 2 conteneurs de communiquer ensemble

Exécutez la commande : `docker network create mon_reseau`

Cela permet d'instancier un nouveau réseau de type bridge sur lequel on va brancher notre conteneur

Lancer un conteneur nginx en spécifiant le réseau sur lequel il doit se brancher

```
docker run -d --name premier_conteneur --network mon_reseau nginx
```

Vous pouvez vérifier que le conteneur est bien connecté au réseau en faisant :

`docker inspect premier_conteneur` -> dans la partie Network

Ou

`docker network inspect mon_reseau` -> dans la partie Containers

### 3. Connecter plusieurs conteneurs en réseau

Maintenant qu'on a créé notre réseau, c'est qui serait fou ce serait de connecter plusieurs conteneurs ensemble non ?

Reprenez la commande du point d'avant en changeant le nom du conteneur.

Exécutez la une deuxième ou une troisième fois si vous le souhaitez, nommez un des conteneurs « secondcontainer »

Vérifiez bien que tous les conteneurs sont sur le même réseau

Plus qu'à tester ! connectez-vous sur un des conteneurs, installez ping si besoin via la commande « `apt-get update && apt-get install iputils-ping` »

Exécutez la commande suivante :

`ping secondcontainer`

Si le ping répond bien, votre installation est correcte félicitations !!

### 4. Déconnecter un conteneur d'un réseau

Pour déconnecter / isoler un conteneur, deux méthodes sont faisables :

- Arrêter et redémarrer le conteneur avec une autre configuration réseau  
`docker stop`  
`docker run -d --name premier_conteneur --network none nginx` -> le none permet d'isoler complètement le conteneur, si on fait un ping dessus, cela ne répondra pas
- Déconnecter le conteneur via la commande  
`docker network disconnect mon_reseau premier_conteneur`

### 5. Faire le ménage

`Docker network prune`

## Exercice 6 – Docker compose

Objectif : Connaître l'utilité d'un docker compose, savoir les manipuler gérer une application dans sa totalité simplement

Docker compose est un outil basé sur Docker permettant de gérer un ensemble de conteneurs à l'aide d'un fichier de configuration -> `docker-compose.yml`

Dans ce fichier sera inscrit tous les services qui sont déployés, avec leur configuration respective, leur interconnexion, leurs dépendances mutuelles etc...

Pour faire simple, c'est un orchestrateur qui automatise le cycle de vie des conteneurs.

### 1. Lancer un premier docker compose

Comme pour le Dockerfile, ouvrez un terminal dans un répertoire de votre choix.

Créez un fichier nommé « docker-compose.yml » -> attention il doit exactement avoir ce nom là

Nous allons lancer un conteneur nginx à partir d'un docker compose, pour cela dans le fichier docker-compose.yml, mettez ces quelques lignes (attention ici également, les indentations sont importantes) :

```
version: "3.8"
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
```

Facile non ? On dit que le service sera nommé nginx, qu'il est basé sur la dernière image de nginx et expose le port 80 mappé sur le port 8080 de l'hôte

Dans le terminal, toujours dans le même répertoire où se trouve votre fichier yml, lancez le docker compose avec la commande :

```
docker compose up
```

## 2. Lancer plusieurs services interconnectés via un réseau

On a dit que le docker compose était un orchestrateur de conteneur, donc il est capable de gérer la mise en place de plusieurs services à la fois, testons ça !

Remplacez le contenu du docker compose par celui-ci :

```
version: "3.8"
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    networks:
      - backend

  redis:
    image: redis:latest
    networks:
      - backend

networks:
  backend:
```

Ici, nous lançons également un conteneur nginx mais cette fois ci avec une bdd redis. Ils sont tous les deux connectés au réseau « backend ».

Testez cette nouvelle configuration avec docker compose up. Pour tester la bonne connexion entre les deux services, connectez-vous sur le conteneur nginx et faites un ping sur redis ☺

### 3. Montage d'un volume dans un docker compose

Dans le fichier précédent, ajoutez une configuration liée au volume

Pour ce faire :

version: "3.8"

services:

  nginx:

    image: nginx:latest

    volumes:

      - ./index.html:/usr/share/nginx/html/index.html

    ports:

      - "8080:80"

    networks:

      - backend

  redis:

    image: redis:latest

    networks:

      - backend

networks:

  backend:

Ici nous synchronisons le fichier index.html du conteneur nginx avec un fichier index.html dans le répertoire courant.

Pour tester que ça fonctionne bien, lancer le docker compose et allez modifier le fichier sur le conteneur.

Vérifiez ensuite que le fichier a été modifié également dans le répertoire courant sur la machine hôte

### 4. Utiliser les variables d'environnements

Maintenant voyons le cas des variables d'environnement. Dans l'utilisation d docker compose, généralement on aime bien avoir des valeurs dynamiques concernant les variables d'environnements, cela permet de faire des modifications à chaud sur les conteneurs au lieu de devoir aller modifier les fichiers de configurations à chaque fois.

Pour variabiliser les variables d'environnements dans le docker compose, il faut créer un fichier caché nommé env, le docker compose saura aller chercher les valeurs qui sont stockées dedans

Donc, il faut créer un fichier « .env »

A l'intérieur, déclarez vos variables sous ce format :

CONTAINER\_NAME=nginx\_container

Maintenant dans le docker compose, pour prendre en compte cette variable, il faut l'injecter de cette manière

version: "3.8"

services:

  nginx:

    image: nginx:latest

    environment :

```
- container_name=${CONTAINER_NAME}
ports:
- "8080:80"
```

Vérifiez que tout fonctionne bien : lancez le docker compose up et connecter vous sur le conteneur nginx.

Exécutez un « printenv » et validez que votre variable d'environnement est bien présente

Voici un deuxième exemple en mettant directement les variables d'environnement dans le docker compose :

```
version: "3.8"
services:
  php:
    image: php:apache
    ports:
      - "8080:80"

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
```

Ici, les variables sont déclarées en dur, cela veut dire que nous devons modifier ce fichier à chaque fois qu'on doit changer de mdp ou user ou autre..

C'est quand même mieux la première solution !

## 5. Mettre des dépendances entre les conteneurs

On sait manipuler la base des docker compose, c'est top ! Néanmoins, il y a une chose qui est un peu gênante, vous sauriez dire ce que c'est ?

Et oui ! Imaginons le docker compose construit un conteneur apache php avant la base de données ou si la bdd n'arrive pas à se lancer, si le php doit piocher des données dans la base pour se lancer, concrètement ça va planter !

Pour régler ce problème, nous pouvons mettre en place une dépendance dans la construction des conteneurs, c'est là toute la force de docker compose

Remplacez le contenu du fichier docker-compose.yml par le suivant (ajout du « depends\_on »):

```
version: "3.8"
services:
  php:
    image: php:apache
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
```

Ici, nous ajoutons un comportement au docker compose qui dit que si le conteneur mysql n'est pas correctement lancé, il n'essaiera pas de construire le conteneur php.

C'est beau, c'est propre 😊

## 6. Gestion de la résilience des conteneurs

Sur le point d'avant, on a vu comment bien gérer son déploiement -> soit toute l'application, soit rien du tout en gérant les conteneurs avec des dépendances.

Mais qu'est-ce qu'on fait si un conteneur crash ?

Nous pouvons indiquer à docker compose ce qu'il doit faire quand un conteneur crash

Pour cela, sur chaque service dans le fichier de configuration, nous pouvons ajouter une option « restart »

Exemple dans un docker-compose.yml :

```
version: "3.8"
services:
  php:
    image: php:apache
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
```

Ici, si notre bdd crash, docker compose fera en sorte de redémarrer le conteneur tant qu'il n'y a pas un de UP, c'est fou !

Nous pouvons également nous assurer qu'un service est bien UP nous même avec une Healthcheck

```
version: "3.8"
services:
  php:
    image: php:apache
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:5.7
    restart: always
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 3
      start_period: 30s
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
```

Ici, nous venons interroger le service mysql en le pinguant, si le ping ne répond pas correctement, nous savons donc qu'il y a un problème car la bdd n'est pas accessible, donc docker compose va redémarrer le conteneur, incroyable !

## 7. Scalabilité du docker compose

Si notre application fait le buzz, il est possible que beaucoup de clients se connectent en même temps sur nos conteneurs, de ce fait il faut savoir gérer la montée en charge .

Pour se faire, nous pouvons gérer des réplicas avec docker compose. Un réplica est une instance de notre conteneur, si nous en mettons plusieurs, la charge sera partagée entre toutes les instances construites

Remplacez le contenu du docker compose par le suivant :

```
version: "3.8"
services:
  nginx:
    image: nginx:latest
    deploy:
      replicas: 3
```

Ici, on construit un conteneur nginx 3 fois, la charge sera donc répartie entre ces 3 instances

#### 8. Analyser ce qu'il se passe dans un docker compose

Certaines commandes nous permettent de suivre nos docker compose pour les manipuler et les analyser

Lister les docker compose : `docker compose ps`

Voir les logs d'un docker compose en temps réel : `docker compose logs -f`

Arrêter les services : `docker compose stop`

N'hésitez pas à faire un `docker compose --help` pour voir l'ensemble des actions possibles 😊